

Unit 1

Introduction to software Engineering: -

Software: -

Software is a program or set of programs containing instructions that provide desired functionality. Engineering is the process of designing and building something that serves a particular purpose and finds a cost-effective solution to problems.

System: -

System software is a set of generalized programs that manage the resources of the computer, such as the central processing unit, communication links, and peripheral devices.

Software Engineering: -

Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

Software Engineering is mainly used for large projects based on software systems rather than single programs or applications.

The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.

Software Engineering ensures that the software that has to be built should be consistent, correct, also on budget, on time, and within the required requirements.

Principles of Software Engineering

1. **Modularity:** Breaking the software into smaller, reusable components that can be developed and tested independently.
2. **Abstraction:** Hiding the implementation details of a component and exposing only the necessary functionality to other parts of the software.

3. **Encapsulation:** Wrapping up the data and functions of an object into a single unit, and protecting the internal state of an object from external modifications.
4. **Reusability:** Creating components that can be used in multiple projects, which can save time and resources.
5. **Maintenance:** Regularly updating and improving the software to fix bugs, add new features, and address security vulnerabilities.
6. **Testing:** Verifying that the software meets its requirements and is free of bugs.
7. **Design Patterns:** Solving recurring problems in software design by providing templates for solving them.
8. **Agile methodologies:** Using iterative and incremental development processes that focus on customer satisfaction, rapid delivery, and flexibility.
9. **Continuous Integration & Deployment:** Continuously integrating the code changes and deploying them into the production environment.

Attributes of Software Engineering

1. **Efficiency:** It provides a measure of the resource requirement of a software product efficiently.
2. **Reliability:** It assures that the product will deliver the same results when used in similar working environment.
3. **Reusability:** This attribute makes sure that the module can be used in multiple applications.
4. **Maintainability:** It is the ability of the software to be modified, repaired, or enhanced easily with changing requirements.

Role of Software

There is a dual role of software in the industry. The first one is as a product and the other one is as a vehicle for delivering the product. We will discuss both of them.

1. As a Product

- It delivers computing potential across networks of Hardware.
- It enables the Hardware to deliver the expected functionality.
- It acts as an information transformer because it produces, manages, acquires, modifies, displays, or transmits information.

2. As a Vehicle for Delivering a Product

- It provides system functionality (e.g., payroll system).
- It controls other software (e.g., an operating system).
- It helps build other software (e.g., software tools).

Objectives of Software Engineering

1. **Maintainability:** It should be feasible for the software to evolve to meet changing requirements.
2. **Efficiency:** The software should not make wasteful use of computing devices such as memory, processor cycles, etc.
3. **Correctness:** A software product is correct if the different requirements specified in the [SRS Document](#) have been correctly implemented.
4. **Reusability:** A software product has good reusability if the different modules of the product can easily be reused to develop new products.
5. **Testability:** Here software facilitates both the establishment of test criteria and the evaluation of the software concerning those criteria.
6. **Reliability:** It is an attribute of software quality. The extent to which a program can be expected to perform its desired function, over an arbitrary time period.
7. **Portability:** In this case, the software can be transferred from one computer system or environment to another.

8. **Adaptability:** In this case, the software allows differing system constraints and the user needs to be satisfied by making changes to the software.
9. **Interoperability:** Capability of 2 or more functional units to process data cooperatively.

Software Characteristic:-

- It is intangible, meaning it cannot be seen or touched.
- It is non-perishable, meaning it does not degrade over time.
- It is easy to replicate, meaning it can be copied and distributed easily.
- It can be complex, meaning it can have many interrelated parts and features.
- It can be difficult to understand and modify, especially for large and complex systems.
- It can be affected by changing requirements, meaning it may need to be updated or modified as the needs of users change.
- It can be impacted by bugs and other issues, meaning it may need to be tested and debugged to ensure it works as intended.

Difference between Software Engineering and Computer Science:-

Parameters	Software Engineering	Computer Science
Definition	Software Engineering majorly defines the architecture and structural properties of Materialistic Engineering, Thermodynamics Computer Organization Architecture, etc.	Computer science involves the study of both principles and the use of computers which usually covers theoretical and practical approaches.

Parameters	Software Engineering	Computer Science
Focus	Software engineering focuses on the development of software applications.	Computer engineering focuses on the design and development of computer hardware and systems.
Scope	Software engineering is concerned with the entire software development process , from conception to maintenance.	Computer engineering is focused on the physical components and systems that make up computers.
Knowledge Area	Software engineering involves areas such as software design , programming, testing , project management, and quality assurance .	Computer engineering, on the other hand, includes areas such as digital circuits, microprocessors, computer architecture, and operating systems.
Applications	Software engineering is used to develop a wide range of software applications, such as desktop and mobile applications, web applications, and embedded systems.	Computer engineering is used to design and develop computer hardware, such as microprocessors, graphics cards, and motherboards.
Skills Required	Software engineers require skills in programming languages, software design patterns, and development frameworks , as well as project management, and have strong problem-solving communication skills.	Computer engineers require skills in digital circuits, computer architecture, and hardware design.
Main Objective	Software engineering applies all the standards and principles of Engineering to design, develop, maintain, test, and evaluate computer software which is also known as the life cycle of Software Development.	Computer science is formed with the collection of Computer Engineering, Computer Science, Information systems, Information Technology, and Software engineering.

Parameters	Software Engineering	Computer Science
Scope of Study and Application	It involves the study and application of software only.	It involves the study and application of software and hardware.
Professionals	Software engineers are programmers who perform various quality assurance techniques to test the general behavior of software.	Students studying Computer Science study the concepts of various subjects like Mathematics, Electrical engineering, Physics, and Management Information systems.
Structured Process	It is a structural process of checking, verifying, and finding the errors and bugs according to the need of software and then providing a solution for removing that bug.	It is not a structural process as everything is to be done in a process and requires proper study before execution.
Areas of Study	Software Engineering involves some areas of study which are Software Development, Software Testing, and Quality Assurance.	Computer Science involves areas of study which are Networking, Artificial Intelligence, Data Base Systems etc.
Project Management	Software engineering is not compulsory project management but knowledge of project management is a plus.	Project management is an internal part of study in Computer Science.
Processes and Methods	Involves applying engineering principles and methods to the software development process.	Combines principles from electrical engineering and computer science to create new computer technologies and systems.
Phases	Typically involves several phases, such as requirements gathering, design, coding, testing, and maintenance.	Involves designing and developing computer hardware, such as computer

Parameters	Software Engineering	Computer Science
		chips and networking technologies.
Tools and Technologies	Examples of tools and technologies used include programming languages, version control systems, and software design patterns.	Examples of tools and technologies used include computer chips, networking technologies, simulation software, and hardware design tools.

Difference between Software Engineering and System Engineering:-

Aspect	SYSTEM ENGINEER	SOFTWARE ENGINEER
Roles and Responsibility	A System Engineer is a person who deals with the overall management of engineering projects during their life cycle (focusing more on physical aspects).	A Software Engineer is a person who deals with the designing and developing good quality of software applications/software products.
Approach	System Engineers follow an interdisciplinary approach governing the total technical and managerial effort required to transform requirements into solutions.	Software Engineers follow a systematic and disciplined approach for software design, development, deployment and maintenance of software applications.
Scope of Work	In general they are concerned with all aspects of computer based system development including hardware, software and process engineering.	In general they are concerned with all aspects of software development, infrastructure, control, applications and databases in the system.
Learning Points	One thing software engineering can learn from system engineering i.e	One thing system engineering can learn from software engineering

Aspect	SYSTEM ENGINEER	SOFTWARE ENGINEER
	Consideration of trade-offs and use of framework methods.	i.e Disciplined approach to cost estimation.
Focus Areas	System engineers mostly focus on users and domains.	Software engineers mostly focus on developing good software.
Methods	Systems Engineering Methods are Stakeholder Analysis, Interface Specification, Design Tradeoffs, Configuration Management, Systematic Verification and Validation, Requirements Engineering etc.	Software Engineering Methods are Modeling, Incremental Verification and Validation, Process Improvement, Model-Driven Development, Agile Methods, Continuous Integration etc.
Interface Management	It ensures correct external interfaces, interfaces among subsystems and software.	It makes interfaces among software module, data and communication path work.
Educational Qualification	System Engineers requires a broader education background like Engineering, Mathematics and Computer science etc.	While Software Engineers requires Computer Science or Computer Engineering background.

Software cost: -

Software Development Cost: -

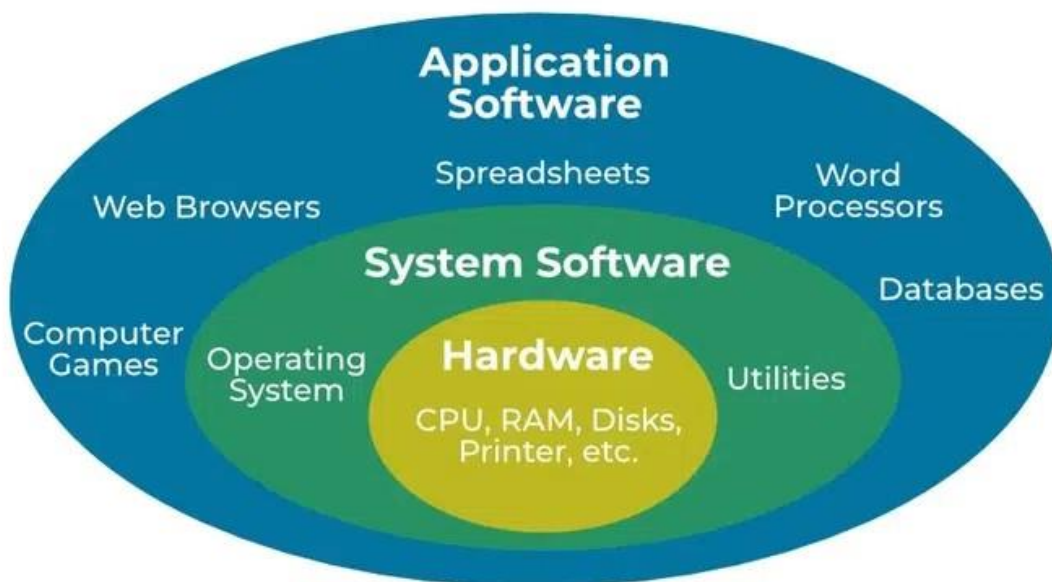
The comprehensive expenditure that occurs from the initial stages of software conception to its final launch. It involves numerous components such as design cost, development cost, testing cost, project management cost, and maintenance cost.

- **Design Cost:** This is the expense required to design the software. It involves cost for wireframing, prototyping and usability testing.

- **Development Cost:** This cost is associated with the actual coding and programming of the software. It generally consumes the most significant portion of the software development cost.
- **Testing Cost:** This is the cost associated with testing the software for bugs and ensuring that it meets the required quality standards.
- **Project Management Cost:** This entails the cost of planning, executing, controlling and closing the software project.
- **Maintenance Cost:** This refers to the cost involved in maintaining the software after it has been launched. It includes costs for updates, fixing bugs, and adding new features.

Software Application:-

The term “application software” refers to software that performs specific functions for a user. When a user interacts directly with a piece of software, it is called application software. The sole purpose of application software is to assist the user in doing specified tasks.



Need for Application Software

End-users can use “application software” to conduct single or many tasks. Following are a few reasons to need application software in computers:

- **Helps the user in completing specified tasks:** Application software is designed with the user in mind. They help the end-user with specialized tasks

in a variety of industries, including education, business, and entertainment. Microsoft Word, for example, is popular application software that allows users to create, edit, delete, and do other tasks with Word documents.

- **Manages and manipulates data:** Business companies utilize application software to manage and manipulate employees, customers, and other databases. Enterprise resource management systems and customer relationship management systems are two common examples of application software.
- **Allows users to effectively organize information:** Individual users can use application software to efficiently create and handle large amounts of data. Microsoft Excel, for example, is popular application software that allows users to manage datasheets.

Types of Application Software

Application software can also be categorized based on its charge ability and accessibility. Here is some application software:

- **Freeware:** It is offered for free, as the name implies. You can utilize freeware application software that you can obtain from the Internet. This software, on the other hand, does not allow you to change it or charge a fee for sharing it. Examples include Adobe PDF, Mozilla Firefox, and Google Chrome.
- **Shareware:** This is given away to users for free as a trial, usually with a limited-time offer. If consumers want to keep using this application software, they will have to pay. WinZip, Anti-virus, and Adobe Reader are instances of shareware.
- **Open-source:** This type of application software comes with the source code, allowing you to edit and even add features to it. These could be offered for free or for a fee. Open-source application software includes Moodle and Apache Web Server.
- **Closed source:** This category includes the majority of the application software programs used nowadays. These are normally charged, and the source code is usually protected by intellectual property rights or patents. It usually comes with a set of restrictions. Microsoft Windows, Adobe Flash Player, WinRAR, macOS, and other operating systems are examples.
- **Word Processing Software:** Word Processing Software can be explained as software that has the functionalities of editing, saving, and creating documents with Word Processor Software like Microsoft Word.

- **Spreadsheet Software:** Spreadsheet Software is a kind of software that deals with the worksheet where it works on some automated version to perform numeric functions. For Example, Microsoft Excel.
- **Presentation Software:** It is a type of application software that is used to present some applications like newly launched functions, products, etc. For Example, Microsoft Powerpoint.
- **Multimedia Software:** Multimedia refers to the mixture of audio, video, image, text, etc., and can be displayed or used with the help of multimedia software. There are so many media players that do this kind of work.
- **Web Browsers:** Web Browser is one of the most used applications worldwide, it takes you to the internet. You can use your desktop, mobile, etc for using this.
- **Educational Software:** Due to the enhancement of the Internet, there are so many educational software runs in the market. It consists of Language learning Software, Classroom Management Software, etc.
- **Graphics Software:** Graphics Software is also used in large amounts. There are so many applications where it is used. Some of the applications include Canva, Adobe, PhotoShop, etc.
- **Simulation Software:** Simulation Software is a kind of Software that is used to compare two different kinds of products and also it helps in evaluating them.

Examples of Application Software

Some examples of application software are:

- **System for Hotel Management:** It relates to the hotel industry's management strategies. Hotel administration, accounting, billing, marketing, housekeeping, and front office or front desk.
- **System for Payroll Management:** It is a term used by all modern businesses to refer to every employee who receives a regular salary or another form of payment. The payroll software calculates all different payment options and generates the relevant paychecks. Employee salary slips can also be printed or sent using this software.
- **System for Human Resources Management:** It describes the systems and activities that exist at the nexus of Human Resource Management (HRM) and Information Technology (IT). The HR department's role is primarily administrative and is found in all businesses.

- **Attendance Recording System:** It's a piece of software that tracks and optimizes a person's or student's presence in an organization or school. Nowadays, customers' existing time/attendance recording devices, such as biometrics/access cards, can be connected with attendance systems. Attendance management can be accomplished in two ways: Integration of biometrics & Integration of manual attendance
- **System of Billing:** It is the billing software that is utilized to complete the billing process. It keeps track of marked products and services given to a single consumer or a group of customers.

Evolution of software Engineering:-

The software evolution process includes fundamental activities of change analysis, release planning, system implementation, and releasing a system to customers.

1. The cost and impact of these changes are accessed to see how much the system is affected by the change and how much it might cost to implement the change.
2. If the proposed changes are accepted, a new release of the software system is planned.
3. During release planning, all the proposed changes (fault repair, adaptation, and new functionality) are considered.
4. A design is then made on which changes to implement in the next version of the system.
5. The process of change implementation is an iteration of the development process where the revisions to the system are designed, implemented, and tested.

Necessity of Software Evolution

Software evaluation is necessary just because of the following reasons:

1. **Change in requirement with time:** With time, the organization's needs and modus Operandi of working could substantially be changed so in this frequently changing time the tools(software) that they are using need to change to maximize the performance.
2. **Environment change:** As the working environment changes the things(tools) that enable us to work in that environment also changes proportionally same happens in the software world as the working environment changes then, the organizations require reintroduction of old software with updated features and functionality to adapt the new environment.
3. **Errors and bugs:** As the age of the deployed software within an organization increases their preciseness or impeccability decrease and the efficiency to bear the increasing complexity workload also continually degrades. So, in that case, it becomes necessary to avoid use of obsolete and aged software. All such obsolete Pieces of software need to undergo the evolution process in order to become robust as per the workload complexity of the current environment.
4. **Security risks:** Using outdated software within an organization may lead you to at the verge of various software-based cyberattacks and could expose your confidential data illegally associated with the software that is in use. So, it becomes necessary to avoid such security breaches through regular assessment of the security patches/modules are used within the software. If the software isn't robust enough to bear the current occurring Cyber attacks so it must be changed (updated).
5. **For having new functionality and features:** In order to increase the performance and fast data processing and other functionalities, an organization need to continuously evolve the software throughout its life cycle so that stakeholders & clients of the product could work efficiently.



Laws used for Software Evolution

1. Law of Continuing Change

This law states that any software system that represents some real-world reality undergoes continuous change or become progressively less useful in that environment.

2. Law of Increasing Complexity

As an evolving program changes, its structure becomes more complex unless effective efforts are made to avoid this phenomenon.

3. Law of Conservation of Organization Stability

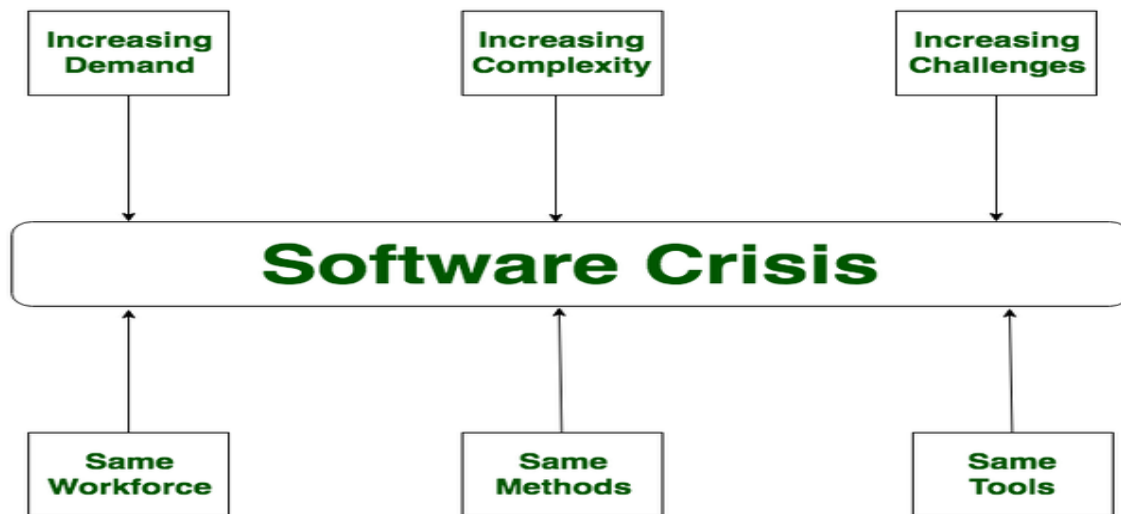
Over the lifetime of a program, the rate of development of that program is approximately constant and independent of the resource devoted to system development.

4. Law of Conservation of Familiarity

This law states that during the active lifetime of the program, changes made in the successive release are almost constant.

Software Crisis-Problem and Causes:-

Software Crisis is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time. The software crisis was due to using the same workforce, same methods, and same tools even though rapidly increasing software demand, the complexity of software, and [software challenges](#). With the increase in software complexity, many software problems arose because existing methods were insufficient.



Causes of Software Crisis

Following are the causes of Software Crisis:

- The cost of owning and maintaining software was as expensive as developing the software.
- At that time Projects were running overtime.
- At that time Software was very inefficient.
- The quality of the software was low quality.
- Software often did not meet user requirements.
- The average software project overshoots its schedule by half.

- At that time Software was never delivered.
- Non-optimal resource utilization.
- Challenging to alter, debug, and enhance.
- The [software complexity](#) is harder to change.

Factors Contributing to Software Crisis

Factor Contributing to Software Crisis are:

- Poor project management.
- Lack of adequate training in software engineering.
- Less skilled project members.
- Low productivity improvements.

Solution of Software Crisis:

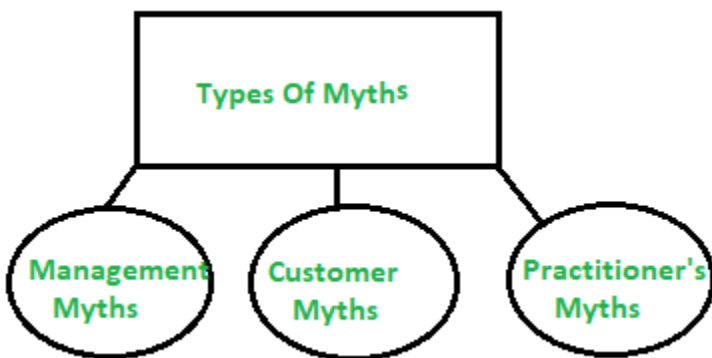
There is no single solution to the crisis. One possible solution to a software crisis is [Software Engineering](#) because software engineering is a systematic, disciplined, and quantifiable approach. For preventing software crises, there are some guidelines:

- Reduction in software over budget.
- The quality of the software must be high.
- Less time is needed for a software project.
- Experienced and skilled people working on the software project.
- Software must be delivered.
- Software must meet user requirements.

Software Myths:-

Software Myths:

Most, experienced experts have seen myths or superstitions (false beliefs or interpretations) or misleading attitudes (naked users) which creates major problems for management and technical people. The types of software-related myths are listed below.

**(i) Management Myths:****Myth 1:**

We have all the standards and procedures available for software development.

Fact:

- Software experts do not know all the requirements for the software development.
- And all existing processes are incomplete as new software development is based on new and different problem.

Myth 2:

The addition of the latest hardware programs will improve the software development.

Fact:

- The role of the latest hardware is not very high on standard software development; instead (CASE) Engineering tools help the computer, they are more important than hardware to produce quality and productivity.
- Hence, the hardware resources are misused.

Myth 3:

- With the addition of more people and program planners to Software development can help meet project deadlines (If lagging behind).

Fact:

- If software is late, adding more people will merely make the problem worse. This is because the people already working on the project now need to spend time educating the newcomers, and are thus taken away from their work.

(ii)Customer Myths:

The customer can be the direct users of the software, the technical team, marketing / sales department, or other company. Customer has myths leading to false expectations (customer) & that's why you create dissatisfaction with the developer.

Myth 1:

A general statement of intent is enough to start writing plans (software development) and details of objectives can be done over time.

Fact:

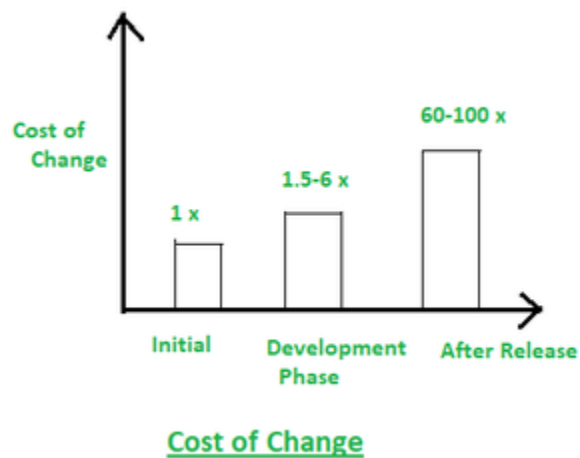
- Official and detailed description of the database function, ethical performance, communication, structural issues and the verification process are important.
- Unambiguous requirements (usually derived iteratively) are developed only through effective and continuous communication between customer and developer.

Myth 2:

Software requirements continually change, but change can be easily accommodated because software is flexible

Fact:

- It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When requirements changes are requested early (before design or code has been started), the cost impact is relatively small.



Different Stages of Myths

(iii) Practitioner's Myths:

Myths 1:

They believe that their work has been completed with the writing of the plan.

Fact:

- It is true that every 60-80% effort goes into the maintenance phase (as of the latter software release). Efforts are required, where the product is available first delivered to customers.

Myths 2:

There is no other way to achieve system quality, until it is "running".

Fact:

- Systematic review of project technology is the quality of effective software verification method. These updates are quality filters and more accessible than test.

Myth 3:

An operating system is the only product that can be successfully exported project.

Fact:

- A working system is not enough, the right document brochures and booklets are also required to provide guidance & software support.

Myth 4:

Engineering software will enable us to build powerful and unnecessary document & always delay us.

Fact:

- Software engineering is not about creating documents. It is about creating a quality product. Better quality leads to reduced rework. And reduced rework results in faster delivery times

Professional and Ethical Responsibility:-

Software engineering involves wider responsibilities than simply the application of technical skills.

Software engineers must behave in an honest and ethically responsible way if they are to be

respected as professionals. Ethical behaviour is more than simply upholding the law.

Issues of professional responsibility**Confidentiality**

- Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

Competence

- Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.

Intellectual property rights

- Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

Computer misuse

- Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

Code of ethics - principles**PUBLIC**

- Software engineers shall act consistently with the public interest.

CLIENT AND EMPLOYER

- Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.

PRODUCT

- Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.

JUDGMENT

- Software engineers shall maintain integrity and independence in their professional judgment.

MANAGEMENT

- Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.

PROFESSION

- Software engineers shall advance the integrity and reputation of the profession consistent with the public interest. ©Ian Sommerville 2004 Software Engineering, 7th edition. Chapter 1 Slide 28

COLLEAGUES

- Software engineers shall be fair to and supportive of their colleagues.

SELF

- Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Software Process:-

Software processes in software engineering refer to the methods and techniques used to develop and maintain software. Some examples of software processes include:

- **Waterfall**: a linear, sequential approach to software development, with distinct phases such as requirements gathering, design, implementation, testing, and maintenance.
- **Agile**: a flexible, iterative approach to software development, with an emphasis on rapid prototyping and continuous delivery.
- **Scrum**: a popular Agile methodology that emphasizes teamwork, iterative development, and a flexible, adaptive approach to planning and management.
- **DevOps**: a set of practices that aims to improve collaboration and communication between development and operations teams, with an emphasis on automating the software delivery process.

Components of Software

There are three main components of the software:

1. **Program**: A computer program is a list of instructions that tell a computer what to do.
2. **Documentation**: Source information about the product contained in design documents, detailed code comments, etc.

3. **Operating Procedures:** Set of step-by-step instructions compiled by an organization to help workers carry out complex routine operations.

Principal of Software Engineering:-

- **KISS (Keep It Simple, Stupid)**
The KISS principle promotes software simplicity by focusing on avoiding pointless complexity and giving priority to crucial features and functionalities.
- **DRY (Don't Repeat Yourself)**
Code repetition that is not necessary should be avoided as recommended by the DRY philosophy. It highlights the value of code reuse and encourages developers to use pre-existing code rather than duplicating it throughout various software system components
- **YAGNI (You Aren't Gonna Need It)**
The YAGNI principle highlights the significance of simply implementing the essential features during software development as opposed to making an effort to foresee and include every potential feature that might be required in the future.
- **BDUF (Big Design Upfront)**
The BDUF principle states that software systems should be designed as completely as possible before any coding begins. While this approach can be useful in some cases, it can also lead to unnecessary complexity and a lack of flexibility.
- **SOLID**
SOLID is an acronym that stands for the five principles of object-oriented design: Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, and Dependency Inversion Principle.
- **Occam's Razor**
Occam's Razor states that all things being equal, the simplest solution is usually the best. This principle encourages developers to avoid unnecessary complexity and to focus on the most straightforward and effective solution to a given problem.
- **Law of Demeter (LoD)**
The Law of Demeter, also known as the principle of least knowledge, states

that software components should only interact with a limited number of other components.

- **Avoid Premature Optimization**

Premature optimization refers to the practice of optimizing the software before it is necessary to do so. This can lead to unnecessary complexity and a lack of flexibility.

- **Measure Twice and Cut Once**

Before starting the coding process, this notion urges engineers to take their time to carefully plan and build software systems.

- **Principle of Least Astonishment**

Software systems should act in a way that is consistent with what the users and stakeholders expect, according to the Principle of Least Astonishment.

Software Quality Factors:-

Software Quality shows how good and reliable a product is. To convey an associate degree example, think about functionally correct software. It performs all functions as laid out in the [SRS document](#). But, it has an associate degree virtually unusable program. even though it should be functionally correct, we tend not to think about it to be a high-quality product.

Factors of Software Quality

The modern read of high-quality associates with software many quality factors like the following:

1. **Portability:** A software is claimed to be transportable, if it may be simply created to figure in several package environments, in several machines, with alternative code merchandise, etc.
2. **Usability:** A software has smart usability if completely different classes of users (i.e. knowledgeable and novice users) will simply invoke the functions of the merchandise.
3. **Reusability:** A software has smart reusability if completely different modules of the merchandise will simply be reused to develop new merchandise.

4. **Correctness:** Software is correct if completely different needs as laid out in the SRS document are properly enforced.
5. **Maintainability:** A software is reparable, if errors may be simply corrected as and once they show up, new functions may be simply added to the merchandise, and therefore the functionalities of the merchandise may be simply changed, etc
6. **Reliability:** Software is more reliable if it has fewer failures. Since software engineers do not deliberately plan for their software to fail, reliability depends on the number and type of mistakes they make. Designers can improve reliability by ensuring the software is easy to implement and change, by testing it thoroughly, and also by ensuring that if failures occur, the system can handle them or can recover easily.
7. **Efficiency.** The more efficient software is, the less it uses of CPU-time, memory, disk space, [network bandwidth](#), and other resources. This is important to customers in order to reduce their costs of running the software, although with today's powerful computers, CPU time, memory and disk usage are less of a concern than in years gone by.

Software Quality Attributes:-

- Availability;
- Correctness;
- Interoperability;
- Modifiability;
- Maintainability;
- Performance;
- Usability;
- Reusability;
- Reliability;
- Security;
- Testability.

Correctness defines the ability of the system to perform tasks as defined by its specification. If the program is consistent with its specification, it is considered to be 100% correct. The system is either correct or not correct at all. Correctness has no middle position. It is an all-or-nothing attribute.

Interoperability measures how well the system interacts with other systems. Or, how easy is it to move your system to another platform?

Maintainability displays how costly it is to keep the system running without breaking down. This property demonstrates whether the changes can be adapted cost-effectively or not.

Usability quality is an attribute of navigation. It measures the user experience while interacting with a system. How long does it take for the system to execute a task? How easy is it for a user to learn to do a task? How many buttons should a person click on to do a certain task? How well can the system meet the user's requirements?

Reusability means different code library classes should be created in a way to be easily used in different program modules. In other words, components in one system can be used as components in another system without a change. It is an important attribute of quality to reduce development costs and time.

Reliability is about how well the system operates over a period of time. The program should work smoothly under different conditions in different environments.

Security displays how the system manages malicious or unauthorized access or modification, which can destroy it and compromise the legitimate user's experience. Security is measured by the ability of the program to operate without risk of harm to users, the software environment, and resources.

Software Engineering Methods:-

Software engineering methods provide an organized, systematic approach for specifying, designing, constructing, testing, and verifying the resulting software products and associated work items involved in developing computer software

applications. The methods impose a certain structure, set of steps, practices, and procedures on the software engineering effort to make it more methodical, repeatable, and ultimately more success-oriented.

What are heuristic methods?

Heuristic methods constitute a category of experience-based software engineering methods that are fairly well-established and widely practiced throughout the software industry. These methods provide rational unified process approaches for developing software using informal sets of “rules of thumb” that have developed over time.

What are formal methods?

Formal methods comprise a specialized class of software engineering methods that apply a rigorous, mathematics-based notation and language for specifying, developing, and verifying computer software. They provide an approach to software construction rooted in mathematical proofs and formal logic rather than empirical heuristics.

Software Process Model:-

The framework activities that are used to all software projects.

It includes following five activities:

- (1) communication
- (2)Planning
- (3)modeling
- (4)construction
- (5)Deployment

This activity framework activity can be use for the development of small simple programs large web application or large computer base system.

(1)COMMUNITION:

Before doing any technical work it is iportant to communicate and collaborate with customer. it is intent to understand the objective of the project and gather the requirement of the project.

Which helps to define the future and function of the software?

Activities->

- 1)Define problem
- 2)Request clarify
- 3)fesibility study
- 4)Request approval
- 5)collect required data.

(2)PLANNING:

This activities estblishes a plan the software development.

It describes the technical tasks to be conducted,consider the risk factor.

The resources that will be require and manage a work schedule.

(3)MODELING:

The software engineering creates the model,using this model developer and the customer to better understand the software requirement,and to make the design of system by collected requirements.

(4)CONSTRUCTION:

The activity combines the code generation and testing. The code generation is either manual or automated. The testing is required uncovering errors in the code.

And here make accurate system.

(5) SOFTWARE DEPLOYMENT:

The software is delivering to the customer who evaluates the delivered product and provides the feedback base on the evaluation.

Here check out the software with new software provides required output compare to old system.

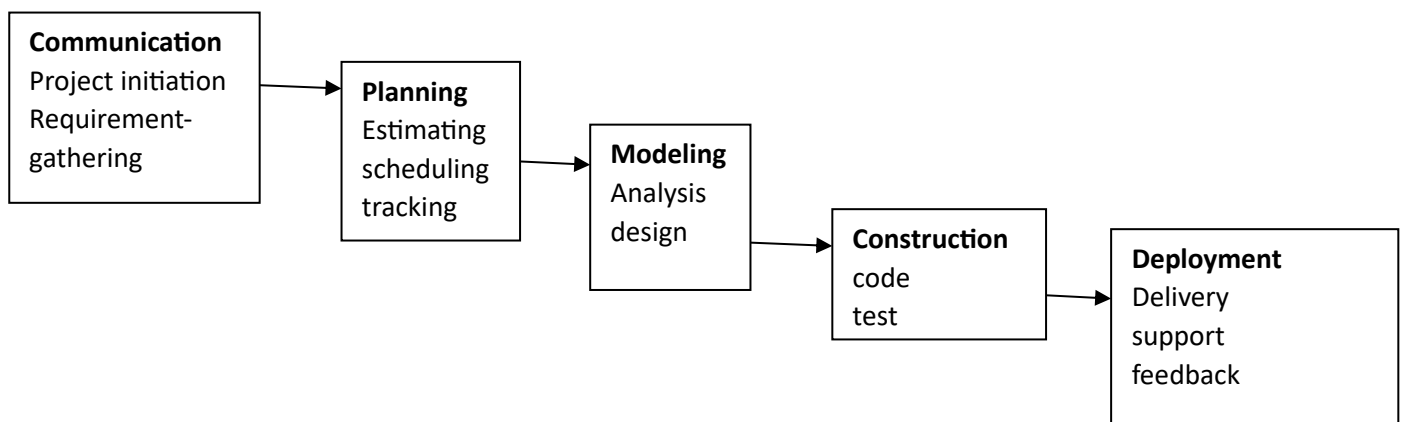
Waterfall Model:-

It is also known as liner model. also called sequential model.

The simplest model

-> Which state that the phases(activity) are organized in liner fashion? It is use when the trequirments of the problem are well understood.

-> In the waterfall model as clc(classic life cycle). Which is suggesting to the systematic sequential approach thta begins with the customer requirement and progress through planning, modeling, construction, and deployment?



->This model is described the relationship quality insurance to action associated with communication, modeling and other activity.

->The project begins with the feasibility analysis and successfully demonstrating the feasibility of the project after completion of feasibility analysis the requirement analysis and project planning begins.

->After completion of the project planning design of the project, then after start the coding.

->once the coding is finish the code integration and testing is done.

->After completion of testing the system is installing and regular operation and maintenance of the system take place.

->Here all the activity can be perform in linear way; linear order of activity has some important consequence.

(1) Clearly identify and of the way designs the new phase.

A) certificate mechanism (v&v validation & verification).

B) Each phase has some define output.

C) Intermediate products in the form of document or report.

(2) Each phase complete and its output is certified. The output of this phase is the input of next phase and it can not be change or modify.

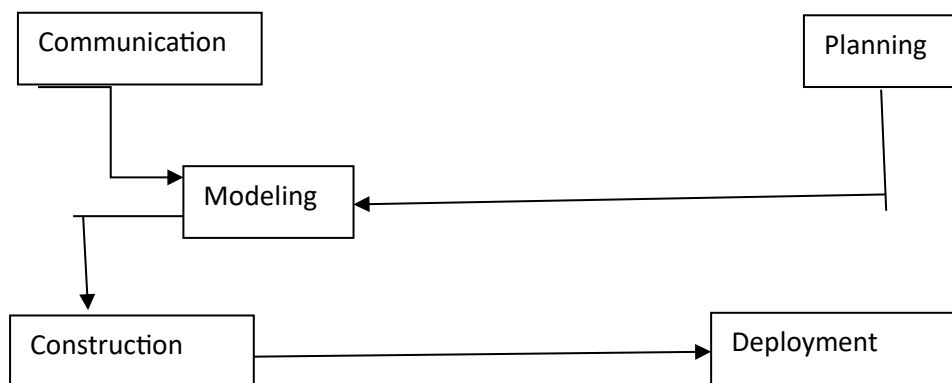
Relation of waterfall model:

Any difference of ordering of the phase will generate less successful product for successful project listed in the waterfall model must be perform in linear way.

Limitation of waterfall model:

- 1) way of adopt (develop)
- 2) freezing the requirement.
- 3) not useful for developing generic product.

Fig: 1) Relation waterfall model



Prototyping Modal:-

- The prototyping modal requires that before carrying out the development of the actual software,
A working prototype usually inefficient, inaccurate, or dummy functions.
- A prototype is usually built using several short cuts.
- The short cuts might involve using inefficient, inaccurate, or dummy functions.

There are several advantages of building a prototype before the actual product is developed

- 1) As important use of a prototype can be to illustrate the input data formats, messages, reports, and the interactive dialogues to the customer.

2) In this regard, the prototype modal turns out to be especially useful in developing the Graphical user interface (GUI) part of a system.

3) Another important situation where the prototyping modal can be useful where the exact technical solutions to be adopted are unclear team.

Here we have to find out technical capability of developer

4) It is useful when we develop the new system.

The prototype modal of software development is shown figure. As shown in figure, the phase is prototype development to control various risks. This is followed by an iterative development cycle. In this modal, prototype starts with an initial requirements gathering phase.

A quick prototype is built. The developed prototype is submitted to the customer for his evaluation. Based on the customer feedback, the requirements are refined and the prototype is suitably modified. This cycle of obtaining customer feedback and modifying the prototype, the actual system is developed using the iterative waterfall approach.

However, the experience gathered from development the prototype help a great deal in developing the actual system.

Therefore, even though the construction of a working prototype might involve additional cost, for system with unclear customer requirements and for systems with unresolved technical issues,

The overall development cost might turn out to be lower in the prototype modal than that of an equivalent system development using the iterative waterfall modal

Limitations:- It is not better for existing system design is carried out and

Incremental Model:-

- Basic idea of increment of modal is adding some functional capability into the system until the whole system is completed.
- User can quickly refine the function (expend or increment the functionality) in later software finish.
- Each step, extension and design modification can upgrade the system.

Advantage

- Result is better than existing system because each increment is physical then testing the entire system.
- The increment modal combines element of linear phase process flow. Each linear sequence produces renewable increment of the software.

Example:- The word processing software develop using increment model if deliver the basic file management, editing and document productions function in the first increment.

- In the second increment in deliver some new functionality spelling and grammar checking and more sophisticated editing and document production.
- In the third increment it at the advance page layout capability.
- In the fourth increment it add the more user friendly environment and increase more functionality in it
- In increment process model focus on the delivery of a product with each increment
- Early increments are down version of the final product.
- It is plan for manage the technical risk.
- It is useful when unavailable for complete implementation by the business headline.

- The core product is well received then the required staff can easily add new functionality in the next increment.

- **Limitation** is it is useful for existing system not for new system.

Spiral Model:-

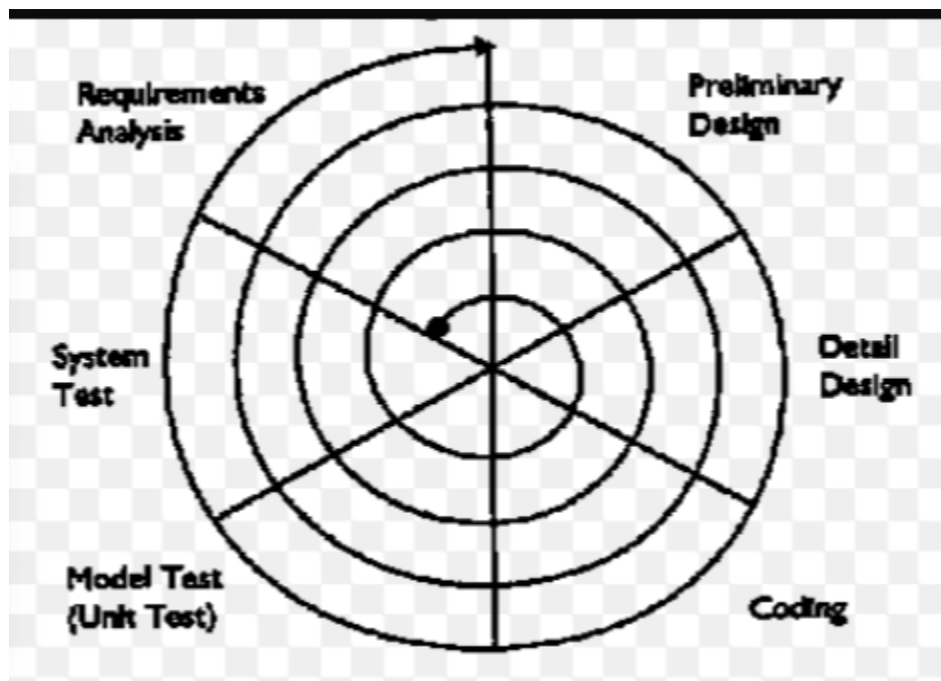
- The spiral model is originally proposed by Barry Boehm, the spiral model is an evolutionary software process model that couples the iterative nature of prototype with the controlled and systematic aspects of the waterfall model.

- The spiral development model is **a risk driven process model** generated that is used to guide multi-stakeholders concurrent engineering of software intensive systems.

- It has two main distinguishing features.

1) one is a cyclic approach for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk.

2) The other is a set of anchor point milestones for ensuring stakeholders' commitment to feasible and mutually satisfactory system solutions.



A spiral model is delivered into set of framework activities defined by the software engineering team.

- Each of the framework activities represent one segment of the spiral path, the software team performs activities that are implied by a circuit around the spiral in clock wise direction,beginning at the center.

- The first circuit around the spiral might result in the development of a product specification subsequent passes around the spiral might be used to develop

A prototype and then progressively more sophisticated versions of the software.

Unlike other process model that end when software is delivered, the spiral model can be adapted to apply throughout the life of the computer software.

The spiral model is a realistic approach to the development of large-scale systems and software.

Because software evolves as the process progresses, the developer and customer better Understand and react to risk at each evolutionary level.

The spiral model uses prototyping as a risk reduction mechanism but,more important,enables you to apply the prototyping approach at any stage in the evolution of the product

The spiral model demands a direct consideration of technical risks at all stages of the project and, if properly applied,should reduce risks before they become problematic